

P2.9b — closure record

Status: closed, 29 September 2026 **Record written:** 6 October 2026 **Source:** reconstructed from the P2.9b working sessions. Where a figure appears here it was reported from a run, not estimated.

1. What P2.9b was

P2.9b was added to scope on 17 September, when the decision was taken to make the loader compatible with **raw LinkedIn exports** rather than only the project's own pre-processed table. That turned a demo into a product: a visitor uploads the file LinkedIn gives them, and the app builds the network in front of them.

The scope revision of 22 September confirmed the dependency order and dropped all day counting and end-dates:

P2.9b → P2.9 → Phase 2 checkpoint → API → MCP server → hosting

P2.9b therefore owned everything that had to be true before the app could be run by somebody who is not the author: an in-process build, a partial-failure story, a way to pay for classification, and a session that ends.

2. Test suite

Point	Tests
P2.9b opened	333
Slice 3 + D-63	561
D-64 (cache isolation)	566
Session expiry + D-65	621
D-66 merge	628
Content-keyed cache	647
P2.9b closed (D-67)	649
D-68	651
D-70 rewording	654 (current)

`check_single_loader` 7/7 throughout. `sweep_count_surfaces --strict` ends at the same three pre-existing findings it started with.

3. What was delivered

3.1 Resource spike

Run on a throwaway branch (`spike/resources`, since deleted) against the synthetic export on Streamlit Community Cloud, Python 3.11.

Measure	Result	Rule
Peak memory	1,332 MB of 3,072 (43%)	≤ 60%
Embedding + two UMAP fits	11.5 s	≤ 60 s

Conclusion: the build runs in the app process. No worker split, runtime lock unchanged. The spike also surfaced D-52 (Community Cloud substitutes pyarrow 24.0.0 for the locked 25.0.0 over a known segfault); 24.0.0 is now pinned.

3.2 Partial-failure semantics — option B

The full network is always shown. Classification that failed produces a fourth “**not classified yet**” state rather than a missing node, carried through every consumer: `build_network()`, `retry_unclassified()`, `session_build.py`, and the notice in `unclassified.py`.

The notice’s wording is reason-specific and never merges two reasons, because each is a false claim in the other’s case:

- `_PHRASE_KEY_FAILED_OWN` — the user’s own key was rejected or ran out of credit
- `_PHRASE_KEY_FAILED_PROJECT` — the classification service was unavailable
- `_PHRASE_KEY_FAILED_UNKNOWN` — “the classification did not complete”, for a build that recorded no source (added by D-63)

3.3 Upload path and welcome screen

`linkedin.py` takes bytes and drops email, URL and date at parse. Keyless build, demo landing, “Start over” in the sidebar. Both routes verified end to end in a browser.

3.4 Synthetic demo

Shipped as a tracked artefact via `tools/build_demo.py`. D-56 fixed along the way: Plotly draws `circle-open` as nothing, so the marker is now `diamond`.

The demo’s classification cache ships tracked, holding the synthetic titles. A clone rebuilding the demo therefore reads the author’s cached classifications rather than drawing fresh ones from a non-deterministic classifier. The guarantee is the cache’s, not the node table’s. Titles that share a normalised key are each sent, and the cache keeps the last answer, which a rebuild then gives to all of them. On 6 October a rebuild differed from the shipped table in one row, a confidence value, with the group and the review count unchanged (D-73).

The shipped demo is run 5 of the five D-69 measurement runs: 124 of 442 needing review (28.05%), generated 29 September. The date is recorded in `data/demo/provenance.json`, and the run is identified in the D-69 commit (95d35e6). It is the highest of the five draws, not the lowest.

3.5 Free tier

`src/spend.py` — `GenerationMeter`, `DailyLedger`, `Meters`, `BudgetExceeded`, `estimate_titles`.

Constant	Value
Measured cost per title	\$0.00095 (100/100 answered, 8,791 in / 7,782 out)
<code>ASSUMED_COST_PER_TITLE</code>	\$0.0012
<code>MAX_SPEND_PER_GENERATION</code>	\$5.00
<code>MAX_SPEND_PER_DAY</code>	\$30.00
<code>FREE_CHAT_QUESTIONS</code>	10
Anthropic Console limit	\$150/month (five saturated days)

The pre-flight estimate **refuses rather than half-spends**. A mid-build stop happens at a batch boundary and writes `REASON_BUDGET_REACHED`. The daily ledger is file-backed at `data/spend/daily.json`, gitignored, atomic writes that fail open on disk errors.

Chat charges the daily ledger only, never the generation meter; the 10-question count is the real user-facing limit. The allowance is enforced **per session, not per day** — session state resets when a tab closes, so “10 questions today” would be a promise the code cannot keep.

Correction note — 7 October 2026 (D-80, D-81). This section describes `src/spend.py` and `chat_allowance.py` accurately, and the app as it was at closure not at all: - Nothing constructed the meters, so no build or retry was capped, the pre-flight never refused, and

`data/spend/daily.json` was never written (D-81, fixed in `2c49ce0`). - The chat panel never called the allowance, so the ten-question limit was neither applied nor shown. Chat charged nothing to the daily ledger, rather than charging it “only” (D-80, fixed in `baf6016`).

Both were wired on 7 October. Chat, summaries and self-enrichment now charge the daily ledger, and only chat questions count against the allowance (design record, 7 October amendment). The text above is left as written.

3.6 Key field

Vocabulary settled to `KEY_USER` / `KEY_PROJECT` / `KEY_NONE`. `keys.py` is the single resolver (D-55). `session.py` is the single session-store accessor (D-59).

- Validated **on paste** — one cheap call per typo, better feedback
- Field clears on success; “Forget my key” available
- `redact()` on every path that shows exception text, because an SDK error can quote the request it failed on and a key in a traceback on a public deploy is unrecoverable
- One component, `key_field.py`, rendered by `render_upload_panel` on both the welcome screen and the sidebar

3.7 Retry offer

A keyless build puts everyone in `REASON_NO_KEY`, which is correctly **not** retryable — retrying with no key fails identically. When a key then arrives, that reason is stale: it described the world at build time.

`src/dashboard/retry_offer.py` is therefore a **separate control with a different trigger** — a key arriving, rather than a reason being retryable. `_NOT_RETRYABLE` stays strict, so nobody with a dead key is given a button that can only fail.

The offer quotes `provenance["uncached_titles"]`, not the headcount. `count_uncached_titles` was hoisted out of the pre-flight branch so it runs on every build and is stored at build time. Someone is about to spend their own money, and “442 people will use your key” when twelve will is the dishonesty the notice module exists to prevent.

3.8 Provenance on screen

`src/dashboard/provenance.py`. The scope record is specific: stated plainly on screen, not in a tooltip.

Two shapes, one question. A `BuildResult` carries this session’s provenance; the demo carries `data/demo/provenance.json`, written by `tools/build_demo.py`. Neither is rendered differently — the reader does not care which path produced the line.

- `demo_line()` surfaces `real_people == 0` (“No real person appears in it”), the strongest privacy statement the app can make, previously sitting in a file nobody read
- `built_line()` says nothing on a keyless build; the notice below already explains the fourth state
- `prompt_version` and `tau` are deliberately **left out** — they serve whoever is debugging a classification, not the person reading it

Reading `data/demo/provenance.json` also closed D-58 for free: `welcome.py` had the figure 442 hardcoded and now takes it from the same file.

3.9 Session lifetime and expiry

`src/dashboard/lifetime.py` (new) holds everything about running down: `expire_if_stale`, `take_expired`, `minutes_warning`, `watch_session`, the snapshot prompt (`work_snapshot`, `unsaved_work`, `should_prompt`, `record_download`) and `render_lifetime_notices`. `session_build.py` keeps the build and the clock primitives.

Setting	Value
LIFETIME	1 hour of inactivity
WARN_BEFORE	20 minutes
Warning buckets	20, 15, 10, 5 minutes

Time is always injected, so no test sleeps. The whole session is cleared on expiry — build, corrections, notes, and the API key — which also fixed the demo-then-upload path. The prompt reads “Download your work before the session ends” and **asks rather than claims**, because `on_click` fires even if the browser save is cancelled.

The privacy line on the welcome screen was updated to match: read in memory, never saved, gone when the tab closes, **and dropped after an hour of inactivity**.

3.10 Content-keyed embeddings cache

The cache keyed on **row count alone**, which caught a differently sized export and nothing else. Re-running after correcting a title, or against different people with the same headcount, returned vectors computed from someone else’s text — and the network was then placed by them, with nothing on screen to say so.

The key is now a hash of `MODEL_NAME` plus every `profile_text` in order, separator between each. Each part earns its place:

- **The model name**, or a file matches across a model change and hands back another model’s vectors, possibly at another width
- **The separator**, or (“ab”, “c”) and (“a”, “bc”) hash alike, so a title edited across a boundary hits a cache built from different text
- **Order**, because `embeddings[i]` must correspond to `people[i]`

It goes in the **filename, not a sidecar**: a sidecar is a second file to keep in step and a comparison that can be got wrong; in the name, a mismatch is simply a miss and a stale file is visibly stale in a listing. `HASH_LENGTH = 16`.

Saves are atomic — written `.partial`, moved with `os.replace` (atomic on Windows for same-volume moves, which is where `data/cache/` sits). The load is wrapped anyway, since `np.load` raises on a truncated file *before* any shape check. The shape check now covers both dimensions, which is what catches a right-named file at another model’s width.

`data/cache/embeddings.npy` predates the keying and is never read again. Gitignored and recomputable; the README’s new “Caches” section says so, and says that nothing removes old keyed files (~650 KB per distinct content at 442 people).

3.11 About-text acceptance check

Carried over from P2.9a. Verified **against the running app**, not from the test suite: the user has no row in the node table, and their About text reaches the chat as its own labelled block.

4. Defects found and closed

P2.9b found **D-53 through D-67**, several of them older bugs the new work walked into. The authoritative table is `docs/defect_register.md` (kept in sync with its PDF). The entries this record can speak to directly:

ID	What it was
D-52	Community Cloud substitutes pyarrow 24.0.0 for locked 25.0.0 — now pinned

ID	What it was
D-53, D-55	Key resolution scattered; <code>keys.py</code> is now the single resolver
D-56	Plotly draws <code>circle-open</code> as nothing — marker changed to <code>diamond</code>
D-57	Upload fix (see register)
D-58	<code>welcome.py</code> hardcoded the 442 figure — now read from demo provenance
D-59	Three private <code>_store()</code> copies — collapsed into <code>session.py</code>
D-61	<code>KEYLESS_NOTE</code> shown unconditionally — <code>CLASSIFIED_NOTE</code> added
D-62	Tests depending on the ambient shell environment
D-63	<code>build_network</code> defaulted <code>source</code> to <code>KEY_PROJECT</code> , recording a specific claim about whose credit paid for every caller that forgot. Default is now <code>None</code> ; the upload panel and the retry both pass the real source
D-64	Tests reached the real classification cache and left a fabricated answer for a title that does not exist. The obvious fix does not work: <code>CACHE_PATH</code> was captured as a <i>default argument</i> in three functions and re-imported into <code>build.py</code> , so a module-attribute patch reached no call site. Resolved at call time instead, plus a session-level hash check on the real cache file
D-65	Work outliving its network — <code>forget_network_work()</code> now drops corrections, notes, chat history, summaries, skips, selection, retry offer, restore report and download snapshot on <code>clear_built()</code> . A retry does not call it, so corrections survive a retry
D-66	Dashboard crashed on every run past the welcome screen — the key field rendered an expander inside an expander in the sidebar
D-67	<code>on_key=st.rerun</code> passed from both <code>app.py</code> and <code>welcome.py</code> , printing “Calling <code>st.rerun()</code> within a callback is a no-op” above the page, including on the first screen a visitor sees

Not sourced in this record: D-54 and D-60. Both are in the register; transcribe them from there rather than from here.

Two closure findings worth carrying forward as method, not just as fixes:

- **D-64’s real discovery** was not the contamination. Tests that classified without a cache path had been reading answers a *previous test run* wrote, so some were passing on cached values rather than on what the fake client returns. That is D-42’s lesson one level up: a cache that silently supplies answers makes the absence of a real answer invisible.
- **D-66 and D-67 both reached main** because fake `st` doubles do not enforce Streamlit’s rules — only `AppTest` does. Smoke tests through the real handler now cover both, and both fail against the old code on exactly the right symptom.

5. After closure — D-68, D-69, D-70

Clearing the classification cache of real job titles ahead of the public repo turned into a measurement session.

D-68 — how the deployed app finds the project key

Reported as a defect: `keys.py` reads only `os.environ`, Community Cloud has no shell, so a deployment would find no project key and offer every visitor the keyless build.

Closed as not a defect as reported. Streamlit loads `.streamlit/secrets.toml` into the environment at startup, so a root-level `ANTHROPIC_API_KEY` already reaches `os.environ.get`. Adding an `st.secrets` read would have introduced an on-page error for no gain.

Three real things came out of the investigation:

1. **A nested secret is not found.** A key under `[anthropic]` never reaches the environment — and grouping secrets under a section is the natural thing to do. Pinned by tests that load a `secrets.toml` the way the server does, so a Streamlit change surfaces in the suite rather than on a deploy.
2. **streamlit run does not read .env.** A reasonable assumption and a false one.
3. **.streamlit/secrets.toml was not gitignored.** A README section pointing people at that file without the rule invites a live key into a repo about to be made public. Now ignored.

Not verified: Community Cloud itself. The evidence covers `streamlit run` and the secrets file it reads. First-deploy check is in the P2.9 gate.

D-69 — the classifier is not deterministic, and nothing said so

Found while clearing the cache. Rebuilding the demo from a cleared cache gave a different answer to the same synthetic file — and the cache had been hiding that since P2.9a, because every rebuild read cached classifications. The classifier looked stable because it was never asked twice.

Five builds of `data/synthetic/Connections.csv`, identical input, locked config E, same model, cache cleared between each:

Run	Needs review	Share
1	97	21.9%
2	106	24.0%
3	114	25.8%
4	106	24.0%
5	124	28.1%

Range 97–124: 27 people, 6.1 percentage points ($27/442 = 6.11$; subtracting the rounded 21.9% from the rounded 28.1% gives 6.2, which is a rounding artefact), mean about 24.8%. A column diff between run 1 and the shipped table showed **68 of 442 titles (15.4%) receiving a different soc_major**, with role byte-identical. Coordinates and all five centrality columns were unchanged — the layout is fully seeded and the variation is entirely in the classification.

What this invalidates:

- The 21.9% figure belonged to the **P2.9a demo**, which was shipped on 24 September and built before the cache was cleared. It was not one of the five runs. It matched the lowest of them (97 needing review), although 68 of its titles were placed in a different group from run 1. It is not a property of the synthetic file. Any statement that the demo understates the real 29.0% “by about 7 points” is wrong; across these runs the gap is 0.9 to 7.1 points.
- **The demo was rebuilt on 29 September** and ships at 124/442 (28.05%), which is run 5, the highest draw. Against the real network’s 29.0% the gap is under one point. That is itself a single-run comparison, and no more stable than its two inputs.

- The P2.9a calibration spent two pre-registered re-tunings moving a figure whose run-to-run spread (6.1pp) is **larger than the miss that stopped the process** (2.1pp outside a 24–34% band). Three of the five runs land in band with no tuning at all. The process was followed correctly; the instrument was noisier than it assumed.

What this does not invalidate: the locked config E decision stands. A 15% run-to-run disagreement on individual titles and a 6pp spread on the aggregate does not move a 60–64% ceiling enough to reopen it, and the abstention-over-fabrication rule is unaffected. But “measured three convergent ways” must now read “measured three convergent ways, **each a single run.**”

The real network’s 29.0% was deliberately **not** re-measured: it would cost real classification of the author’s own network and would not change any decision. Better to state the uncertainty than chase a second point estimate.

`docs/p2_9a_calibration.md` keeps its record and carries a dated correction note.

D-70 — the enrichment panel attributes causes it cannot support

The “compare the two readings” panel implied that a difference between the title-only reading and the About-informed reading came from the user’s description. It cannot support that, for three separate reasons:

1. **Each reading is a single call.** `classify_title_only` runs with `use_cache=False` and `classify_from_about` has no cache, so the screen compares one draw with one draw. Pressing the button again can give a different result.
2. **The prompts differ, and that confound predates D-69.** The enriched reading uses `self_enrichment.py`’s own prompt with rules 2, 3 and 7 added. Even a perfectly deterministic classifier could differ because it was asked a different question. The feature presents “same classifier, more words” and is actually “different prompt, more words.” Noise makes attribution *uncertain*; the prompt difference makes it *systematically confounded*.
3. **The 15% figure does not transfer.** It comes from the batched title tier on the demo. This path — single call, prose prompt, no cache — is unmeasured.

Done: the wording now reports what each reading *shows* and names no cause. Where the readings differ: “The two readings use different prompts and each is a single answer, so a difference between them may not come from your description.” Where they agree, the matching form: “...so their agreeing does not show your description made no difference.”

Outstanding: run `compare()` a handful of times on one real title through the CLI to get this path’s own spread. Carried into the P2.9 open items.

6. Decisions locked during P2.9b

Decision	Resolution
Partial failure	Option B — full network, fourth “not classified yet” state, retry button
Key validation	On paste
Pasted key + existing keyless build	Offer retro-classification, do not do it automatically
Cost quoted to the user	From <code>provenance["uncached_titles"]</code> , never the headcount
Chat allowance scope	Per session, not per day
Chat spend	Charges the daily ledger only
Session lifetime	1 hour inactivity, whole session cleared including the key
Embeddings cache key	Content + model name, in the filename

Decision	Resolution
Demo classification cache	Ships tracked, so a rebuild reads these classifications rather than drawing fresh ones (not table-identical — D-73)
Public repo	Fresh repo, option 2 (no history); current repo stays private as the archive

Correction note — 7 October 2026 (D-80, D-81). The “Chat spend” row was not true of the code at closure: chat charged nothing, and no cap was enforced. See the note in §3.5. The row is now true, as decided, and the free-tier caps are enforced.